

Build Assignment — Multiplayer Room with an AI Participant

Role: Full Stack Engineering Intern **Timebox:** 5–6 hours. Please don't exceed it; we'd rather see honest scope-cutting than a rushed, feature-complete version. **Format:** A working repo (frontend + backend) you can run locally, a README, and a short NOTES.md.

What Ziroo is

Ziroo is an AI colleague for small teams. You ask it something in plain language and it does the work: reads your inbox, preps a meeting, pulls your analytics, builds a deck. It already knows the company and has the team's tools connected.

This assignment is about a smaller, harder version of the same problem: **an AI agent that has to work correctly when more than one person is talking to it in the same place at the same time.**

The scenario

Two people share a room. Both can type at once. Both can address an AI agent that's a participant in that room, not a private assistant each of them gets alone.

User A: "@agent summarize the last PR discussion." User B, a few seconds later: "@agent what's blocking the deploy?"

The agent has to answer each of them correctly, without mixing up who asked what, even though both questions arrived close together in the same shared stream.

What to build

1. **A shared room.** Two users join (a simple name + room code is enough, no real auth needed) and see each other's messages arrive live. WebSockets, SSE, or polling, your choice, say why you picked it in the README.
2. **An AI agent as a third participant in the room.** It responds when addressed (e.g. `@agent ...`), not to every message.
3. **Context isolation.** The agent must keep each user's thread of conversation straight. User A's question and its context must never leak into the answer given to User B, even though

they share a room.

4. **Some persistence.** Messages and room state should survive a server restart, or you document clearly why you chose not to persist and what the tradeoff was.
5. **A real UI.** Message list, per-user identity, agent messages visually distinct from human ones, a loading/typing state while the agent is thinking, a reasonable empty state for a fresh room.

Constraints worth designing against

- **You do not need a scalable multiplayer server.** Pick the simplest transport that actually works for two users and justify it.
- **Keep the AI call cheap.** One call per addressed message, not per keystroke, not a multi-stage pipeline. We're not evaluating how good the model's answer is, we're evaluating what you built around the call.
- **Assume both users can type at the same time.** Think about message ordering and race conditions, and say what you did about them.
- **Assume the LLM call can fail, time out, or return junk.** The agent should degrade gracefully (say so in the room), never crash it or hang it for both users.

What we're evaluating

In roughly this order:

1. **Does the agent actually keep two users' context straight.** This is the core test of the assignment, not a nice-to-have. A submission that works for one user and quietly breaks with two is a fail on the thing we're actually testing.
2. **Backend judgment.** How you modeled room/session state, what you did about concurrency, what you persisted and why.
3. **AI integration hygiene.** Whether you validate or defensively handle a malformed, slow, or failed model response, or just trust it and hope.
4. **Frontend craft.** Real interaction states UI/UX, not a functional list of divs, this is the surface we can actually compare across candidates.
5. **Honesty in your NOTES.md.** What you verified vs. what you're assuming would work, what you'd improve with more time, and why you made the transport/persistence choices you did.

Practical notes

-
- Use whatever AI API/key you already have (OpenAI, Gemini, Anthropic, whatever's easiest for you). We are not evaluating which model you picked.
 - You do not need a paid plan. Keep your own token usage light while developing, a handful of test messages is enough to prove it works.
 - Two browser tabs (or two devices) are enough to represent the two users. No real auth system required.
 - Questions are welcome and count in your favor, working out what's ambiguous is part of the problem, we've deliberately left some things unspecified.
 - Include a NOTES.md: what you'd improve with more time, what you tested vs. assumed, and any tradeoffs you made under time pressure.
 - Use python, fastapi backend. frontend -react
-

Submission

- A repo link (invite us if private) or a zip.
- A README with setup instructions: it should run with a documented `install` + `dev` command and state any env vars needed (e.g. your own API key, never send us a key)